NoSQL

Lesson 2

Mongo Shell

- Start mongo shell
- Type the following statement in the shell:

```
2024-01-09T07:59:3
> show dbs
Lang        0.000GB
Vaccine     0.000GB
admin       0.000GB
bookdb      0.000GB
```

```
> use bookdb
switched to db bookdb
> show collections
book
>
```

Mongo Shell

- Start mongo shell
- Type the following statement in the shell:

```
> use bookdb
switched to db bookdb
```

```
> show collections
book
>
```

Mongo Shell

- Find documents

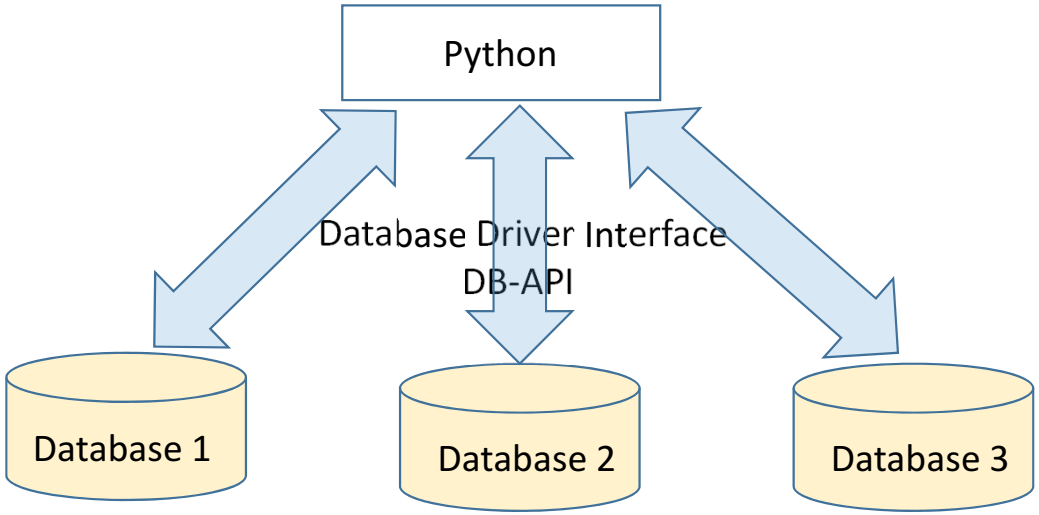
```
>
> db.book.find({}).pretty()
```

Mongo Shell

- Result of find({})

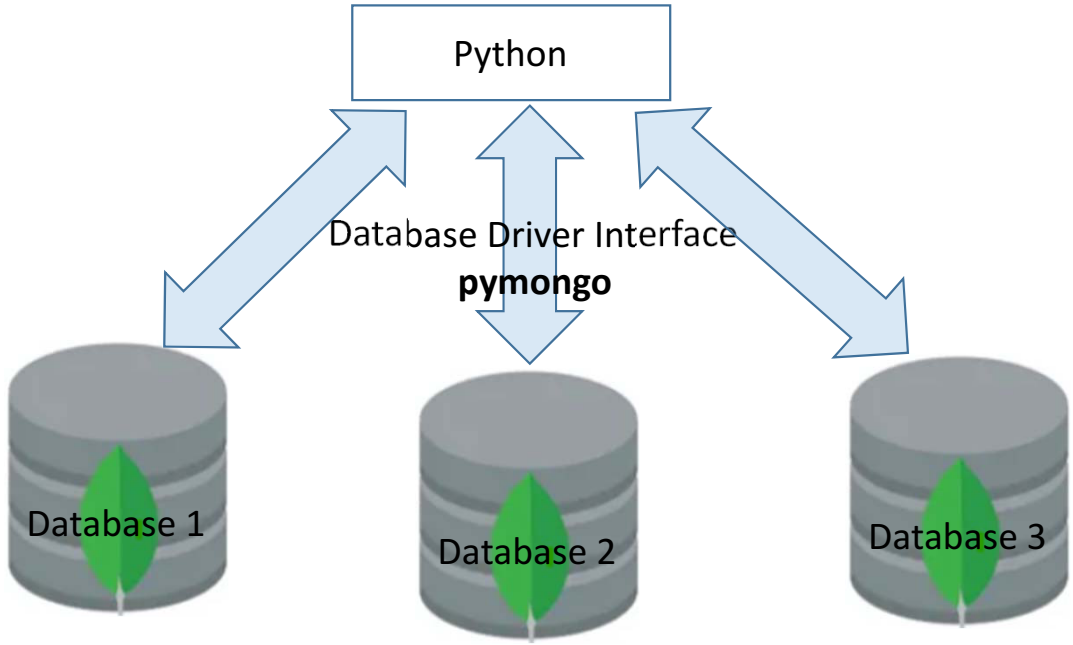
```
>
>
> db.book.find({}).pretty()
{
        "_id" : 8,
        "title" : "Flex on Java",
        "isbn" : "1933988797",
        "pageCount" : 265,
        "publishedDate" : ISODate("2010-10-15T07:00:00Z"),
        "thumbnailUrl" : "https://s3.amazonaws.com/AKIAJC5RLADLUMVRPFDQ.book-thumb-images/allmon.jpg",
        "shortDescription" : "   A beautifully written book that is a must have for every Java Developer.       Ashish K
ulkarni, Technical Director, E-Business Software Solutions Ltd.",
        "longDescription" : "In the demo, a hip designer, a sharply-dressed marketer, and a smiling, relaxed developer s
ip lattes and calmly discuss how Flex is going to make customers happy and shorten the workday   all while boosting the
bottom line. The software systems they're using have been carefully selected and built from the ground up to work togeth
er seamlessly. There are no legacy systems, data, or competing business concerns to manage.    Cut to reality.    You're
 a Java developer. The marketing guy tells you that \"corporate\" wants a Flex-based site and you have to deliver it on
top of what you already have. Your budget  Don't even ask. \"Make it look like the Discovery channel or something.\"
Flex on Java assumes you live in the real world   not the demo. This unique book shows you how to refactor an existing w
eb application using the server-side you already know. You'll learn to use Flex 3 in concert with Spring, EJB 3, POJOs,
JMS, and other standard technologies. Wherever possible, the examples use free or open source software.    The authors s
tart with a typical Java web app and show you how to add a rich Flex interface. You also learn how to integrate Flex int
o your server-side Java via the BlazeDS framework, Adobe's open-source remoting and web messaging technology for Flex.
  The book shows you how to deploy to not only the web but also to the desktop using the Adobe Integrated Runtime (AIR).
 You will learn how to integrate Flex into your existing applications in order to build a next generation application th
at will delight users.    Flex on Java is approachable for anyone beginning Java and Flex development.    ",
        "status" : "PUBLISH",
        "authors" : [
                "Bernerd Allmon",
```

Overview – DB-API



Database	Python Package
Oracle	cx_oracle, pyodbc
SQL Server	pymssql, pyodbc
PostgreSQL	psycopg2
MySQL	MySQL Connector/Python, pymysql

pymongo



https://pymongo.readthedocs.io/en/3.13.0/

Using pymongo

- import pymongo
- Connect to MongoDB at ‘localhost’ port 27017

```
##import pymongo
from pymongo import MongoClient
client = MongoClient('localhost', 27017)
```

Connection to a database

- Query the customers collection in ’mydatabase’ database
- Using the method count_documents to return the number of documents in the collection.

```
db = client.mydatabase
db.customers.count_documents({})
```

```
mydb = client["mydatabase"]
coll = mydb["customers"]
```

Read data – all (no condition)

db.collection.find(query)

```
db.coll.find( {} )
```

To find all documents in collection

```
documents = coll.find({})
for doc in documents:
       print(doc)
```

Read data – with condition/s

```
db.coll.find( {"name":"Ken"})
```

Show documents with name that contains 'Ken'

```
query = {"name": "Ken"}
documents = coll.find(query)
for doc in documents:
       print(doc)
```

Read data – with condition/s

```
db.coll.find( {"name":"Ken", "class": "S66"} )
```

With multiple queries

Show documents with name that equals 'Ken' **AND** class that equals 'S66'

```
query = {"name": "Ken", "class":"S66"}
documents = coll.find(query)
for doc in documents:
      print(doc)
```

Read data

db.collection.find(query, projection)

Projection follows some basic rules:

- The `_id` field is always included unless explicitly excluded.
- Specifying a field for inclusion implicitly excludes all other fields except the '_id' field.

```
db.coll.find({"class":"S66"},{"name":1, "class":1})
```

query

projection

Show name **and** class in document with query in person collection

```
query = {"class":"S66"}
proj = {"name":1, "class":1}

documents = coll.find(query, proj)
for doc in documents:
      print(doc)
```

Read data using Query with Projection

Projection follows some basic rules:

- The `_id` field is always included unless explicitly excluded.
- Specifying a field for inclusion implicitly excludes all other fields except the '_id' field.

```
db.coll.find({"class":"S66"},{"name":0})
```

query projection

Show all fields **except name** in document with query in person collection

```
query = {"class":"S66"}
proj = {"name":0}

documents = coll.find(query, proj)
for doc in documents:
       print(doc)
```

Read data using Operators

- Comparison Query Operators

Name	Description
$eq	Matches values that are equal to a specified value.
$gt	Matches values that are greater than a specified value.
$gte	Matches values that are greater than or equal to a specified value.
$in	Matches any of the values specified in an array.
$lt	Matches values that are less than a specified value.
$lte	Matches values that are less than or equal to a specified value.
$ne	Matches all values that are not equal to a specified value.
$nin	Matches none of the values specified in an array.

Read data using Operators

Equal to

```
documents = coll.find({"qty": {"$eq":10 } })
```

Find documents with qty = 10

Greater than

```
documents = coll.find({"qty": {"$gt":10 } })
```

Less than or equal to

```
documents = coll.find({"qty": {"$lte":10 } })
```

Not equal

```
documents = coll.find({"qty": {"$ne":10 } })
```

Read data using Operators

Find a document with class in S66, S60

$in

```
documents = coll.find({"class":{"$in":["S66", "S60"]}})
```

Find a document with class not in S62, S63

$nin

```
documents = coll.find({"class":{"$nin":["S62", "S63"]}})
```

Read data (previously …)

```
db.coll.find( {"name":"Ken", "class": "S66"} )
```

With multiple queries

Show documents with name that equals 'Ken' **AND** class that equals 'S66'

```
query = {"name": "Ken", "class":"S66"}
documents = coll.find(query)
for doc in documents:
      print(doc)
```

Read data using Compounded Queries

Find a document with class = S66 **AND** name = "Ken"

$and

```
documents = coll.find({"$and":[{"class":"S66"},{"name":"Ken"}]})

documents = coll.find({"$and":[{"class":{"$eq":"S66"}},{"name":{"$eq":"Ken"}}]})
```

Find a document with class = S60 **OR** name = 'Ian'

$or

```
documents = coll.find({"$or":[{"class":"S60"},{"name":"Ian"}]})

documents = coll.find({"$or":[{"class":{"$eq":"S60"}},{"name":{"$eq":"Ian"}}]})
```

Read data using $exists operator

The $exists operator matches documents that contain or do not contain a specified field, including documents where the field value is null.

$exists

```
documents = coll.find({"qty":{"$exists":True}})
```

Find documents with qty field

Find documents with qty field and qty in (5, 10)

```
documents = coll.find({"qty":{"$exists":True, "$in":[5,10]}})
```

Some useful functions

The limit() method returns the stated number of documents stated with the limit() method

```
limit()
documents = coll.find({"qty":{"$exists":True}}).limit(3)
```

Find documents (limited to 3) with qty field

The sort() method sort by fields in ascending or descending order

```
sort()
query = {"qty":{"$exists":True}}
documents = coll.find(query).sort([("name",1),("qty", -1)])
```

Find documents with qty field and sort by name (ascending) and qty (descending order)

Embedded Document

Embedded documents are documents that are nested within other documents. It allows complex data structures to be represented in a single document.
An example of an document with embedded document

```
{"name":
    {
        "given": "Jeanne",
        "family": "Ang"
    }
},
{"name":
    {
        "given": "John",
        "family": "Tan"
    }
}
```

Query Exact Matches on Embedded Document

```
db.person.find({"name":{"given": "Jeanne", "family":
"Ang"})
```

OR

```
db.person.find({"name.given" : "Jeanne"})
db.person.find({"name.family" : "Ang"})
```

Array Elements

An example of an document with array elements

```
{"name":"June", "class":"21S67",
"subjects":["GP", "CP", "PH", "MA"]}
{"name": "Amy", "class":"21S67",
"subjects":["GP", "CP", "CH", "MA"]},
{name: "Anthony", class:"21S66",
"subjects":["GP", "CH", "PH", "MA"]}
```

Query for an Array Element

```
db.person.find({"subjects" : "CP"})
```

returns documents where array field **subjects** contains the element "CP"

```
db.person.find({"subjects" : ["CP"]})
```

returns documents where array field **subjects** contains only "CP" element

```
db.person.find({"subjects" : {$in:["CP","PH"]} })
```

returns documents where array field **subjects** contains the element "CP" **or** "PH"

Array of Embedded Documents

```
{ "name" : "Blue", class:"S66",
  "vaccination" :
  [
   {
    "date":"2021-05-22T05:00:00.000",
    "type":"RNA",
    "manufacturer": "Pfizer-BioNTech“
   },
   {
    "date": "2022-05-22T05:00:00.000",
    "type": "RNA",
    "manufacturer": "Moderna"
   }
  ]
}
```

Query for an Array of Embedded Documents

```
db.person.find({"vaccination.manufacturer" :
"Moderna"})
```

returns documents where **vaccination** array contains the element with **manufacturer** field equals "Moderna"

Query for an Array of Embedded Documents

Use the **$elemMatch** operator to specify multiple criteria on an array element.

```
db.person.find({"vaccination":  {"$elemMatch":
{"date":{"$lte": new Date("2022-06-01")}, "manufacturer":
"Pfizer-BioNTech" }}})
```

returns documents where **vaccination** array contains at least one element with both the **date** field less than or equal to 2022-06-01 and the manufacturer = "Pfizer-BioNTech"